AegisHire — Project Definition & Implementation Blueprint

> **Autonomous AI HR & Technical Interview Intelligence Platform**

Table of Contents

---

1. Project Overview

AegisHire is an enterprise-grade, multi-agent AI platform that transforms the technical hiring pipeline. It leverages real developer data (Git repositories, code contributions, commit history) to build rich candidate profiles, generate personalized interview experiences, and provide HR teams with transparent, explainable, and bias-aware hiring intelligence.

Core Pillars

Pillar	Description
Intelligence	AI-driven analysis of candidate repositories, skills, and code quality
Personalization	Dynamically generated interview questions tailored to each candidate's actual work
Integrity	Anomaly detection during live coding interviews (probabilistic, not deterministic)
Transparency	Every AI decision is explainable, auditable, and logged
Modularity	Agent-based architecture allowing incremental feature rollout

What AegisHire is NOT

- It is **not a surveillance tool** — it uses probabilistic risk scoring, never binary verdicts.
- It is **not a replacement for human judgment** — it augments HR decision-making with data.

- It is **not an autonomous production patcher** (in V1) — code suggestions require human validation.

---

2. Problem Statement

Technical hiring today suffers from several systemic issues:

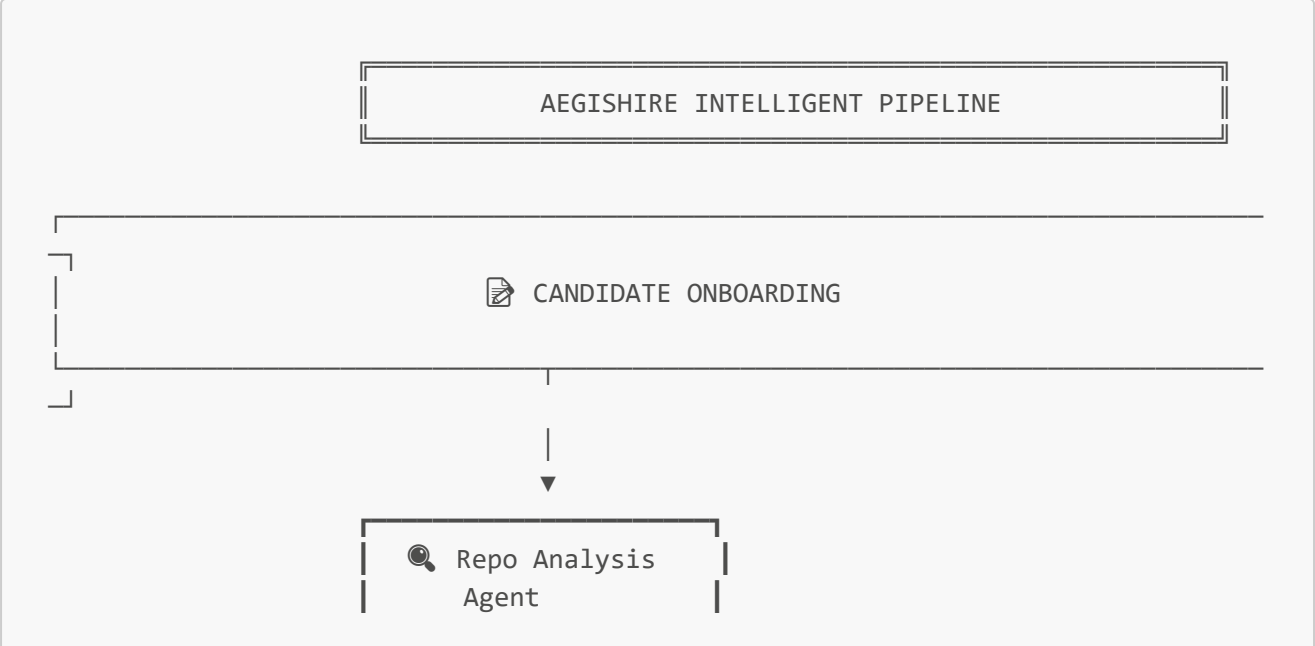
Problem	Impact
Resume-driven hiring	Resumes don't reflect real coding ability. Candidates can fabricate experience.
Generic interview questions	One-size-fits-all questions fail to assess a candidate's actual strengths.
AI-assisted cheating	With LLMs widely available, candidates can generate answers in real-time during interviews. There's no reliable detection mechanism.
Bias in evaluation	Human interviewers introduce unconscious bias. AI systems can amplify it if unchecked.
Lack of transparency	Candidates receive rejection with no explanation. Enterprises can't audit hiring decisions.
Fragmented tooling	Companies use separate tools for screening, interviewing, coding tests, and analytics with no unified pipeline.

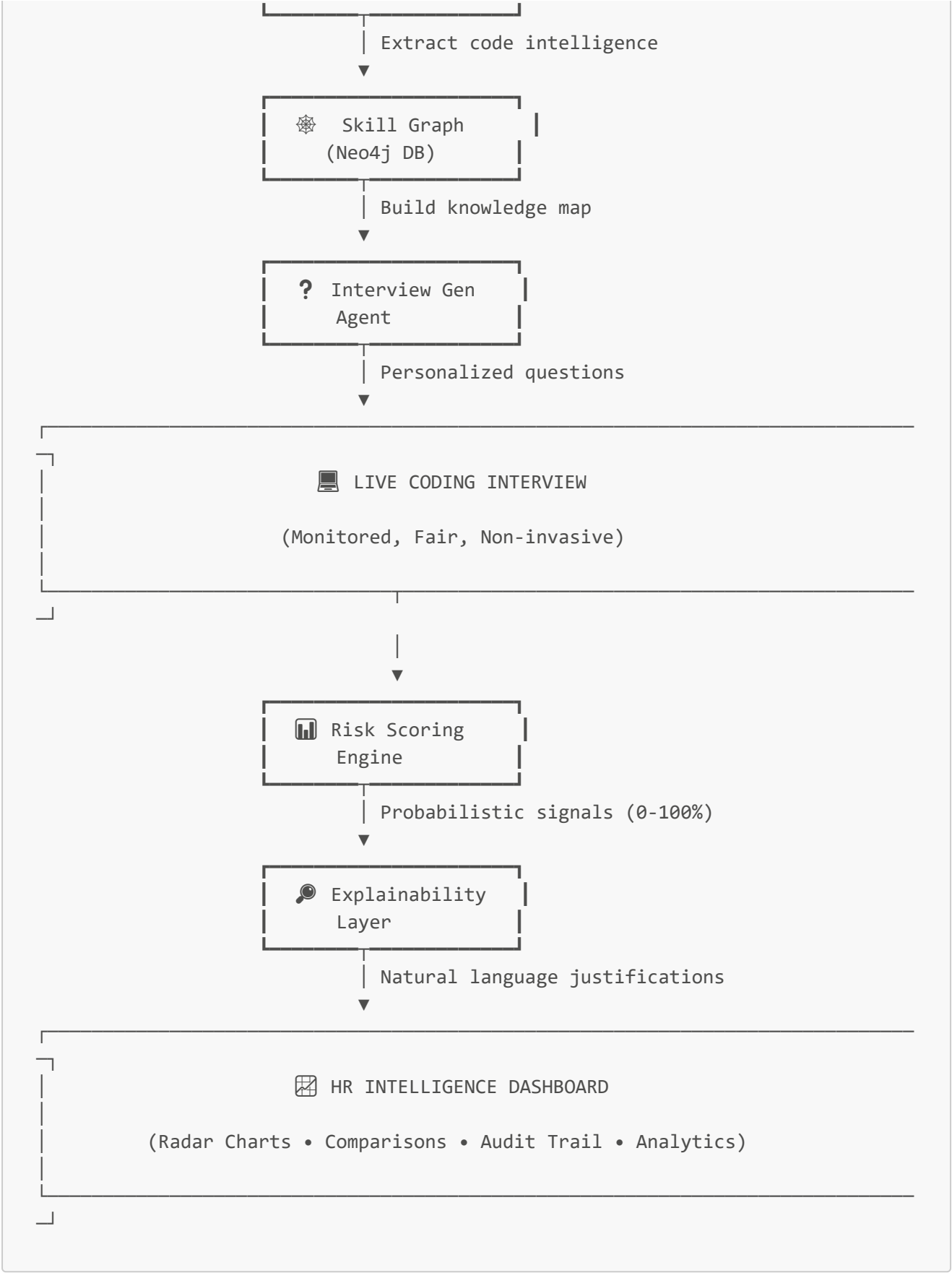
AegisHire addresses all six problems in a single integrated platform.

---

3. Proposed Solution

AegisHire introduces an end-to-end AI-augmented hiring pipeline:





Flow:

1. Candidate links their GitHub/GitLab profile.
2. The **Repository Analysis Agent** extracts tech stack, code quality, commit patterns, architecture style, test coverage, and security awareness.

3. A **Developer Skill Graph** is built in Neo4j — a rich, queryable representation of the candidate's abilities.
4. The **Interview Generation Agent** creates personalized questions based on the skill graph, targeting weak areas and probing strengths.
5. The candidate enters a **Live Code Editor** where typing behavior is passively monitored for anomalies.
6. All signals are aggregated into a **Risk Probability Index** (0–100%) — never a binary "cheated/didn't cheat."
7. HR receives the full candidate profile on an **Intelligence Dashboard** with explainable scoring, radar charts, and comparison tools.

---

4. System Modules & Features

Module A: Candidate Intelligence Engine

Purpose: Build a deep, data-driven profile of each candidate from their real code contributions.

Inputs:

- GitHub / GitLab repository URLs (public or authorized private)
- Candidate-provided metadata (optional)

Analysis Dimensions:

Dimension	What is Extracted	How
Tech Stack	Languages, frameworks, libraries	Dependency file parsing (`package.json`, `requirements.txt`, `Cargo.toml`, etc.) + file extension analysis
Code Quality	Complexity, readability, naming conventions	AST analysis, cyclomatic complexity, linting rule violations
Architecture	Design patterns, folder structure, separation of concerns	Directory tree analysis, import graph construction
Commit Patterns	Frequency, message quality, branch strategy	Git log analysis, conventional commit detection
Testing	Test coverage, test types (unit/integration/e2e)	Test file detection, coverage report parsing, test framework identification
Security	Dependency vulnerabilities, secret exposure, input validation	SAST scanning, dependency audit, regex-based secret detection
Collaboration	PR reviews, issue participation, open-source contributions	GitHub/GitLab API analysis

Output: A structured **Developer Skill Graph** stored in Neo4j.

Module B: Adaptive Interview Generation

Purpose: Generate interview questions that are personalized to the candidate's actual skill profile.

Key Behaviors:

- Questions are derived from the candidate's own repositories (e.g., "In your `auth-service` repo, you used JWT with HS256 — when would RS256 be more appropriate?")
- Difficulty adapts dynamically: if a candidate answers easily, complexity increases
- Questions span categories: conceptual, practical, debugging, system design, and code review
- Each question is tagged with the skill node it targets in the graph

Question Types:

Type	Example
Code Review	"Here's a snippet from your repo. What would you refactor and why?"
Conceptual Probe	"You used the Observer pattern here — when would it fail at scale?"
Debugging Scenario	"This function from your project has a concurrency bug. Find it."
System Design	"How would you redesign your `payment-service` to handle 10x traffic?"
Gap Assessment	"Your repos show no testing. Walk me through how you'd add tests to this module."

Module C: Live Interview Environment

Purpose: Provide a monitored, fair coding environment during the interview.

Components:

C1. Smart Code Editor (Browser-based)

- Syntax highlighting, autocomplete (controlled), and execution sandbox
- Passive behavioral monitoring:
  - Typing speed and rhythm analysis
  - Copy-paste event detection
  - Tab-switch / focus-loss detection
  - Time-to-first-keystroke after question display
  - Burst typing pattern detection (possible AI paste)

C2. Anomaly Scoring Engine

- All behavioral signals feed into a lightweight ML model
- Produces a **Risk Probability Index** per question and per session

- Categories: `Low Risk (0–30)` | `Medium Risk (31–60)` | `High Risk (61–100)`
- Never produces a binary "cheating" flag — always probabilistic
- HR sees the index with full signal breakdown

C3. Optional Voice Channel (V2+)

- Whisper-based speech-to-text transcription
- Voice consistency analysis (same speaker throughout)
- Pause pattern analysis
- This module is **opt-in and consent-required**

Module D: Enterprise HR Dashboard

Purpose: Give HR teams actionable, explainable, and comparable candidate intelligence.

Features:

- **Candidate Radar Chart:** Visual skill profile across 6–8 dimensions
- **Comparison View:** Side-by-side candidate comparison on matching criteria
- **Interview Playback:** Recorded sessions with timestamped annotations
- **Explainability Panel:** For each score, a natural-language explanation of contributing factors
- **Strictness Configuration:** Enterprise-level control over how aggressively anomalies are flagged
- **Historical Analytics:** Hiring funnel metrics, time-to-hire, quality-of-hire tracking
- **Export & Integration:** PDF reports, ATS (Applicant Tracking System) webhook integration

---

Module E: Code Audit Agent (V2+)

Purpose: AI-powered code review for enterprise development teams (post-hire).

Safer Implementation (vs. original auto-patching proposal):

```
Developer pushes commit
        │
        ▼
┌──────────────────┐
│  AI Code Review  │──── Detects vulnerabilities, anti-patterns, performance
issues
└──────────────────┘
        │
        ▼
┌──────────────────┐
│  Suggestion Engine│──── Generates fix suggestions with explanations
└──────────────────┘
        │
        ▼
┌──────────────────┐
│  Staging Auto-   │──── Applies patch in staging/shadow environment
│  Apply (Optional) │
```

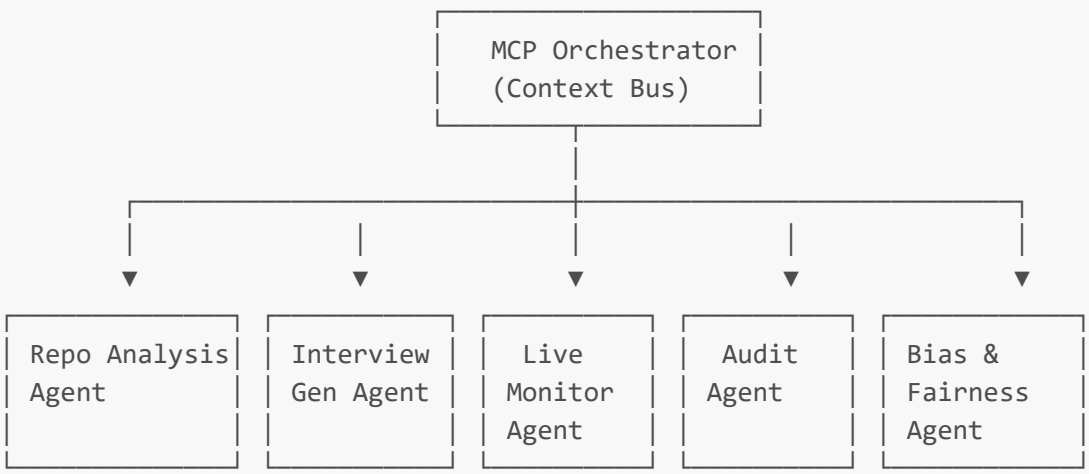
Critical Rule: No AI modification reaches production without human approval and passing CI/CD checks.

---

5. Multi-Agent Architecture (MCP)

AegisHire uses the **Model Context Protocol (MCP)** as its agent orchestration layer. Each agent is an autonomous unit with a specific responsibility, communicating through a shared context bus.

Agent Inventory



Agent	Responsibility	Input	Output
Repo Analysis Agent	Parse Git repos, extract metadata, build skill graph	Git URLs, API tokens	Skill graph nodes + edges in Neo4j
Interview Generation Agent	Create adaptive, personalized questions	Skill graph, job requirements	Ordered question set with metadata
Live Monitoring Agent	Track behavioral signals during interview	Keystroke events, clipboard events, focus events	Anomaly signals, risk scores

Agent	Responsibility	Input	Output
Audit & Transparency Agent	Log every AI decision with explanation	All agent outputs	Audit trail, explainability records
Bias & Fairness Agent	Check for discriminatory patterns in scoring	Evaluation results, demographic data (anonymized)	Fairness report, bias alerts
Code Audit Agent (V2)	Review code commits for issues	Git diffs, repo context	Vulnerability reports, fix suggestions
Cost Controller Agent	Route LLM calls efficiently, track token usage	All LLM requests	Cost reports, model routing decisions

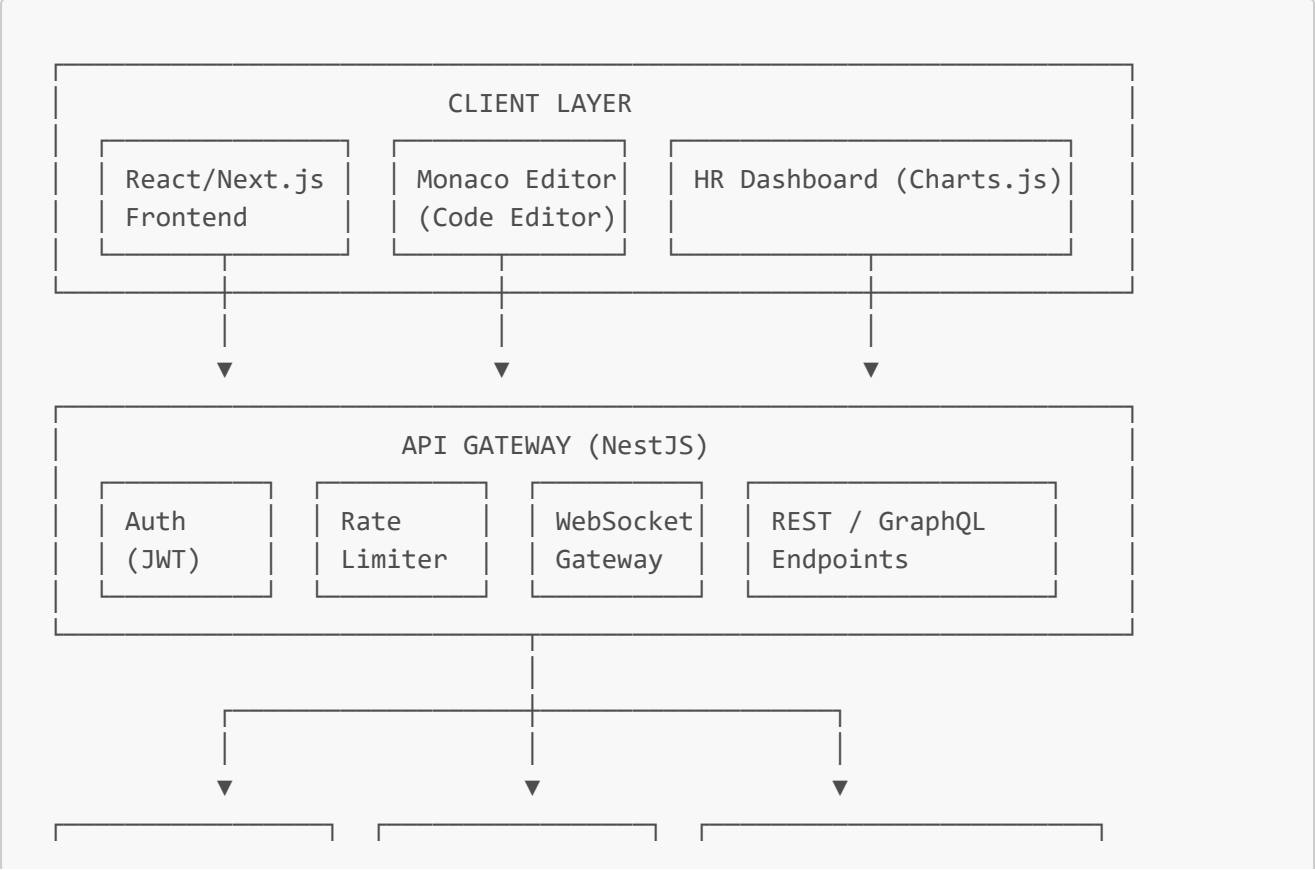
Agent Communication Pattern

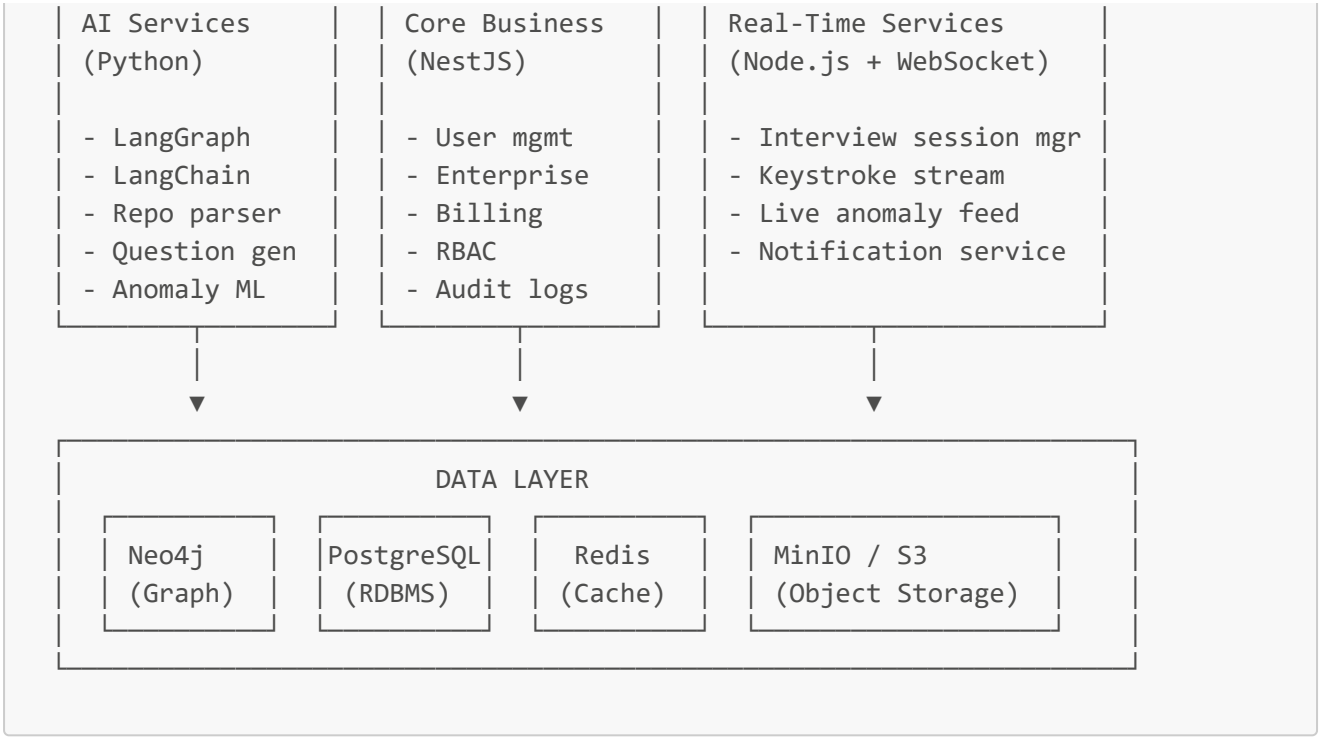
- Agents communicate through the **MCP Context Bus** — a shared state layer
- Each agent reads from and writes to the shared context
- The **Orchestrator** manages agent lifecycle, sequencing, and error handling
- LangGraph manages agent state machines and deterministic workflow transitions
- Agents can be deployed independently (microservice per agent)

---

6. Technical Stack & Architecture

System Architecture Diagram





Technology Choices

Layer	Technology	Justification
Frontend	Next.js (React) + TailwindCSS	SSR for dashboard performance, component ecosystem
Code Editor	Monaco Editor (VS Code engine)	Industry-standard browser code editor with extension support
API Gateway	NestJS (TypeScript)	Modular, decorator-based, excellent for microservice orchestration
AI Services	Python (FastAPI)	LangGraph/LangChain ecosystem is Python-native
Agent Orchestration	LangGraph + MCP	State machine workflows, deterministic agent transitions
Graph DB	Neo4j	Ideal for skill graphs, knowledge graphs, relationship queries
RDBMS	PostgreSQL	Transactional data, user accounts, billing, audit logs
Cache	Redis	Real-time interview state, session management, rate limiting
Object Storage	MinIO (self-hosted) or S3	Interview recordings, logs, generated reports
Message Queue	RabbitMQ or NATS	Inter-service communication, event-driven architecture
Containerization	Docker + Docker Compose	Development and deployment consistency

Layer	Technology	Justification
Orchestration	Kubernetes (production)	Scalability, agent auto-scaling
CI/CD	GitHub Actions	Automated testing, linting, deployment
Monitoring	Prometheus + Grafana	System health, LLM cost tracking, agent performance

LLM Strategy

Use Case	Model	Reason
Code analysis & reasoning	Claude 3.5 / GPT-4o	High reasoning quality for code understanding
Question generation	GPT-4o-mini / Claude Haiku	Good quality at lower cost for templated generation
Speech-to-text	Whisper (local or API)	Best open-source STT model
Embedding (for RAG)	`text-embedding-3-small`	Cost-effective for semantic search
Anomaly classification	Fine-tuned local model	Latency-sensitive, must run in real-time
Summary generation	GPT-4o-mini	Cost-effective for report generation

Cost Control: A **Model Router** dynamically selects the cheapest adequate model per request. Results are cached aggressively. Token budgets are enforced per tenant.

---

7. Data Architecture

Neo4j Skill Graph Schema

```
(Candidate)-[:HAS_REPO]->(Repository)
(Repository)-[:USES_TECH]->(Technology {name, category, proficiency})
(Repository)-[:HAS_PATTERN]->(ArchPattern {name, confidence})
(Repository)-[:HAS_METRIC]->(CodeMetric {complexity, coverage, quality_score})
(Candidate)-[:HAS_SKILL]->(Skill {name, level, evidence_count})
(Skill)-[:RELATED_TO]->(Skill)
(Interview)-[:TARGETS_SKILL]->(Skill)
(Interview)-[:ASKED]->(Question {text, difficulty, category})
(Question)-[:ANSWERED_BY]->(Answer {text, score, time_taken, risk_index})
```

PostgreSQL Schema (Core Tables)

```
candidates       - id, email, name, github_url, created_at, status
enterprises      - id, name, plan, strictness_level, config
interviews       - id, candidate_id, enterprise_id, status, scheduled_at,
```

```
completed_at
interview_sessions - id, interview_id, start_time, end_time, recording_url
evaluations      - id, interview_id, overall_score, risk_index, explanation
audit_logs       - id, agent_name, action, input_hash, output_hash, timestamp,
explanation
users            - id, enterprise_id, role, email, password_hash
billing          - id, enterprise_id, plan, token_usage, cost, period
```

Redis Keys

```
session:{interview_id}        → Live interview state (JSON)
keystrokes:{interview_id}     → Keystroke event stream (Sorted Set)
anomaly:{interview_id}        → Real-time anomaly scores (Hash)
cache:repo_analysis:{repo_hash} → Cached repo analysis results (String, TTL
24h)
rate_limit:{tenant_id}        → API rate limiting (String, TTL)
```

---

8. Implementation Roadmap

Phase 1 — Foundation (Weeks 1–6) — **MVP**

Week	Deliverable
1–2	Project scaffolding: NestJS API, Next.js frontend, Docker Compose, PostgreSQL + Neo4j setup
2–3	GitHub/GitLab OAuth integration, repository connection flow
3–4	**Repository Analysis Agent** — parse repos, extract tech stack, code metrics, build basic skill graph in Neo4j
4–5	**Interview Generation Agent** — generate personalized questions from skill graph
5–6	Basic HR Dashboard — candidate profile view, skill radar chart, question review

Milestone: A recruiter can connect a candidate's GitHub, see their skill profile, and get a set of personalized interview questions.

Phase 2 — Live Interview (Weeks 7–12)

Week	Deliverable
7–8	Monaco-based code editor with sandbox execution (WebSocket-based)
8–9	Keystroke stream capture, copy-paste detection, focus monitoring
9–10	Anomaly scoring model (rule-based first, ML later)
10–11	Real-time risk index display on HR dashboard

Week	Deliverable
11–12	Interview session recording, playback, and annotation

Milestone: Full live coding interview with passive behavioral monitoring and probabilistic risk scoring.

Phase 3 — Enterprise Features (Weeks 13–18)

Week	Deliverable
13–14	Multi-tenant architecture, RBAC, enterprise onboarding
14–15	Strictness configuration, candidate comparison view
15–16	Audit trail system, explainability panel
16–17	Bias & fairness agent — statistical analysis of evaluations
17–18	Billing integration, usage dashboards, PDF report export

Milestone: Enterprise-ready platform with multi-tenancy, audit compliance, and billing.

Phase 4 — Advanced AI & Scale (Weeks 19–24)

Week	Deliverable
19–20	LangGraph-based agent orchestration with full MCP integration
20–21	Adaptive difficulty engine (questions adjust in real-time based on answers)
21–22	Cost controller agent, model routing, caching optimization
22–23	Voice channel (opt-in), Whisper integration, voice analysis
24	Load testing, security audit, documentation

Milestone: Production-grade, scalable, cost-optimized platform.

Phase 5 — Code Audit Module (V2 — Future)

- Code review agent on push events
- Suggestion engine with fix generation
- Staging auto-apply with mandatory human approval
- Integration with CI/CD pipelines

---

9. Improvements & Original Ideas

Beyond the original proposal, here are concrete improvements and new ideas:

9.1. Candidate Self-Assessment Portal

Idea: Before the interview, give candidates access to a **self-assessment view** of their analyzed profile. Let them see what the AI found, correct misattributions (e.g., "I didn't write that code, it was a group project"), and add context.

Why: This solves trust and fairness issues. Candidates feel respected, and the data quality improves. It also protects you legally — the candidate has seen and acknowledged the analysis.

9.2. Interview Simulation Mode

Idea: Offer candidates a **practice mode** where they can experience the interview format, code editor, and question style beforehand — without monitoring or scoring.

Why: Reduces anxiety-driven false positives in anomaly detection. A nervous candidate typing erratically shouldn't be flagged as suspicious. Practice mode establishes their baseline behavior.

9.3. Differential Skill Analysis

Idea: Instead of just analyzing a candidate's repos, compare their skill graph against the **ideal skill profile for the role**. The interview then focuses on the **gap between the candidate and the role requirements**.

```
Candidate Skills:  [React: 85, Node: 70, Docker: 30, SQL: 60, Testing: 20]
Role Requires:     [React: 80, Node: 75, Docker: 60, SQL: 70, Testing: 60]
Gap Analysis:      [React: ✓,  Node: -5, Docker: -30, SQL: -10, Testing: -40]
Interview Focus:   Docker, Testing, SQL (largest gaps)
```

Why: This makes interviews highly efficient. No wasting time on things the candidate already proves through their code.

9.4. Open-Source Contribution Scoring

Idea: Weight open-source contributions differently from personal projects. Contributing to established repos (PR reviews, issue fixes, feature additions) demonstrates collaboration, code review skills, and ability to work in existing codebases.

Why: A candidate who has merged PRs into popular open-source projects demonstrates real-world engineering skills that personal toy projects don't capture.

9.5. Temporal Skill Evolution

Idea: Track how a candidate's skills have **evolved over time** by analyzing commit history chronologically. Show a timeline: "Candidate learned TypeScript in 2023, adopted testing in 2024, started using Docker in 2025."

Why: Growth trajectory is as important as current skill level. A candidate who is rapidly learning is more valuable than one who peaked years ago.

9.6. Interview Knowledge Graph (Cross-Candidate Learning)

Idea: Build a **global knowledge graph** of questions, answers, and outcomes across all interviews. Over time, this enables:

- Identifying which questions best predict job success
- Detecting question fatigue (leaked questions)
- Auto-retiring questions that no longer differentiate candidates

Why: The platform gets smarter with every interview conducted. This is a massive competitive moat.

9.7. Peer Calibration System

Idea: Periodically have **multiple AI agents independently evaluate the same candidate**, then compare their assessments. If agents disagree significantly, flag the evaluation for human review.

Why: Reduces single-point-of-failure in AI evaluation. Simulates the "panel interview" approach where multiple perspectives lead to better decisions.

9.8. Contextual Hint System

Idea: During live coding, instead of letting candidates flounder (which produces poor signal), offer **progressive hints** and measure how many hints are needed. This is a better signal than pass/fail.

```
Question posed → Candidate struggles (2 min no progress)
  → Hint Level 1: "Consider the time complexity"
  → Hint Level 2: "A hash map might help here"
  → Hint Level 3: "Here's the approach: ..."

Score = weighted by hints used:
  0 hints → 100%  |  1 hint → 80%  |  2 hints → 50%  |  3 hints → 20%
```

Why: Measures problem-solving ability with assistance, which is closer to real-world engineering where you Google things and ask colleagues.

9.9. Sandbox Container Per Interview

Idea: Each interview session spins up an **isolated Docker container** with language-specific tooling. The candidate codes inside this container via the browser editor. This enables:

- Real code execution with proper runtime
- Secure isolation (no access to external resources)
- Custom environments per role (e.g., a Python data science env vs. a Java Spring env)

Why: More realistic than a sandboxed editor. Prevents candidates from accessing external help. Provides actual execution results.

9.10. AI Guardrails Layer

Idea: Every LLM output goes through a **validation pipeline** before being shown to users:

```
LLM Response → Bias Check → Factuality Check → Toxicity Filter → Format Validator → Output
```

- Bias Check: Ensures questions don't discriminate based on cultural background
- Factuality Check: Verifies technical accuracy of generated questions/explanations
- Toxicity Filter: Prevents inappropriate content
- Format Validator: Ensures output matches expected schema

Why: LLMs hallucinate. In a hiring context, a hallucinated technical "fact" in a question could unfairly penalize a candidate. Guardrails are non-negotiable.

---

10. Risk Analysis & Mitigation

Technical Risks

Risk	Likelihood	Impact	Mitigation
LLM hallucination in question generation	High	Medium	Guardrails layer, human review queue for flagged questions
Anomaly detection false positives	High	High	Probabilistic scoring only, baseline calibration, practice mode
Neo4j query performance at scale	Medium	Medium	Query optimization, read replicas, pagination
LLM API cost explosion	High	High	Model router, aggressive caching, token budgets per tenant
Agent coordination failures	Medium	High	LangGraph state machines, dead letter queues, circuit breakers
Data breach (candidate PII)	Low	Critical	Encryption at rest/transit, minimal PII storage, SOC2 alignment

Business Risks

Risk	Mitigation

Risk	Mitigation
Legal challenges from monitoring	Consent-first architecture, probabilistic scoring, lawyer review
Enterprise adoption resistance	Explainability panel, bias reporting, compliance documentation
Competitor landscape (HackerRank, CodeSignal)	Differentiate on personalization from real repos (unique moat)
AI model deprecation/API changes	Abstract LLM layer behind interface, support multi-provider

Ethical Risks

Risk	Mitigation
Surveillance perception	Transparent about what is monitored, candidate can see their own data
Algorithmic bias amplification	Bias agent, fairness metrics, regular third-party audits
Over-reliance on AI scores	Position as decision-support, never decision-maker

11. Ethical & Legal Framework

Principles

1. **Consent is mandatory.** No monitoring without explicit, informed, revocable consent.
2. **Transparency is non-negotiable.** Candidates see what data is collected and how it's used.
3. **Probability, not verdict.** Never binary "cheated" / "didn't cheat." Always a risk score with explanation.
4. **Right to explanation.** Every score has a human-readable justification.
5. **Right to deletion.** GDPR Article 17 compliance — candidates can request full data deletion.
6. **Bias auditing.** Regular fairness analysis across gender, ethnicity, age, and geography.

Compliance Checklist

Regulation	Requirement	Implementation
GDPR (EU)	Data minimization, right to deletion, consent	Privacy-by-design architecture, consent service, data retention policies
AI Act (EU)	High-risk AI system transparency	Explainability layer, human oversight, risk documentation
CCPA (California)	Consumer data rights	Data access API for candidates, opt-out mechanism

Regulation	Requirement	Implementation
EEOC (US)	Non-discriminatory hiring	Bias agent, fairness reporting, adverse impact analysis

12. Business Model

Tiered Enterprise Plans

Plan	Monthly Price	Features
Starter	$299/mo	Repo analysis (10 candidates/mo), basic question generation, skill radar
Professional	$999/mo	+ Live code editor with anomaly scoring, interview recording, 50 candidates/mo
Enterprise	$2,999/mo	+ Voice analysis (opt-in), candidate comparison, HR analytics, 200 candidates/mo
Elite	Custom	+ Code audit agent, custom integrations, dedicated support, unlimited candidates

Revenue Channels

1. **Subscription (primary):** Tiered monthly/annual plans
2. **Usage-based add-on:** Extra candidates beyond plan limit ($15/candidate)
3. **API access:** Third-party integration with existing ATS platforms
4. **Professional services:** Custom agent development, training, onboarding

Unit Economics Consideration

Cost Driver	Estimate per Interview	Mitigation
LLM API calls (repo analysis)	$0.50–$2.00	Caching, model routing
LLM API calls (question gen)	$0.10–$0.30	Template-based generation with LLM refinement
LLM API calls (during interview)	$0.20–$0.50	Selective triggering, not continuous
Infrastructure (per session)	$0.05–$0.15	Auto-scaling, spot instances
Total per interview	**~$1.00–$3.00**	Target 10:1 revenue-to-cost ratio

13. Success Metrics

Product Metrics

Metric	Target	Measurement
Repo analysis accuracy	>85% skill extraction precision	Validated against self-reported skills
Question relevance score	>4.0/5.0 (HR rating)	Post-interview feedback survey
Anomaly detection precision	>70% true positive rate	Validated against known cheating scenarios
False positive rate	<15%	Candidates flagged who were confirmed clean
Interview completion rate	>90%	Sessions completed vs. started
Time-to-evaluate	<5 minutes post-interview	Automated scoring pipeline latency

Business Metrics

Metric	Target (Year 1)
Paying enterprises	20+
Monthly interviews conducted	1,000+
Net Promoter Score (HR users)	>40
Monthly Recurring Revenue	$50K+
Churn rate	<5% monthly

Appendix: Key Technical Decisions Log

Decision	Choice	Rationale
Graph DB for skills	Neo4j	Skill relationships are inherently graph-structured; Cypher queries enable complex traversals
API Gateway	NestJS	TypeScript, decorator-based DI, built-in microservice support, mature ecosystem
AI Framework	LangGraph	State machine model matches agent workflow needs; better than raw LangChain for orchestration
Code Editor	Monaco	Same engine as VS Code; rich API for keystroke monitoring
No auto-patching in V1	Safety	AI-generated patches in production carry excessive risk; staging-only with human approval
Probabilistic scoring only	Legal/ethical	Binary cheating detection is unreliable and legally dangerous

Decision	Choice	Rationale
Multi-model LLM strategy	Cost/quality	Expensive models for complex reasoning, cheap models for routine tasks

This document serves as the comprehensive project definition for AegisHire. It should be treated as a living document, updated as implementation progresses and requirements evolve.